Above average 85

Software Engineering Assignment — Spring 2017
'2048' Implementation

James Pollard

February 2017

1 Design Motivations

Before any code was written for the assignment, some very general implementation guidelines were drawn up:

1. The game state/logic and control flow should be as far removed from each other as possible.
2. It should be possible for the game state/logic to be repackaged as a library with minimal development effort.
3. The game state/logic should be extensible to boards with a size greater than four.

It was felt that these guidelines help ensure that the code would be well written: the separation between game state/logic and the program control flow should prevent overcomplicated interdependence between many functions and variables, instead ensuring that well defined 'contracts' are established which make for cleaner code.

The decision to make the game extensible to boards with a size greater than four was made with the intent to ensure that the game algorithms were designed in as general a manner as possible, as this would probably lead to a more elegant solution.

2 Additional Features

Some additional features were proposed and added to the specification: this was not intended simply to make the game more convincing or interesting to play, but rather to ensure that certain user inputs would not cause unintended or undefined behaviour:

- The game should reject a configuration file which indicates the presence of entries which are not (positive) powers of two.

- The game should reject a configuration file which does not contain the right number of entries for the specified board size.

Either of these cases (particularly the second) could easily cause the program to crash or enter undefined behaviour if not checked for, and these checks do not alter the program's behaviour when the user input is correctly crafted.

3 Initial Design

The first design work involved a very general definition of a class to represent the game state called `board`. This class would have at least the following public functions:

`move(...)` a function accepting some kind of argument specifying direction to cause the game state to be changed reflecting a move in the aforementioned direction

`getBoard()` a function accepting no arguments which returns some representation of the game state allowing its caller to print some representation of the game board

`isGameOver()` a function accepting no arguments which returns some value indicating whether game state represents a game for which there is no possible move

These definitions were intentionally very vague: this was intended to prevent the design becoming to rigid at such an early stage in case it turned out the chosen design had serious limitations later on.

4 Game State Implementation

After the very informal definition of the contracts the class would have, the implementation for the game state was worked out.

It was decided that a single vector would be used to represent the game board. The use of a vector allows functions such as `std::vector::push_back` to be used, simplifying the code. It also allowed the board size to be set dynamically at run time without having to dynamically allocate and manage memory: although this feature isn't actually used in the program, it makes the implementation more generalisable.

A variable would also be used to keep track of the number of zeroes present in the grid, which allows addition of a random two to be added more efficiently.

5 Algorithm Design

5.1 Random Two

The addition of a random two to an empty position on the board is a task which must be carried out frequently during the game. One algorithm to carry out this task might randomly 'guess' a number between one and the number of entries on the board and check if the entry on the board represented by this random number is a zero or not. If the entry is a zero, it changes it two a two, otherwise it repeats, until a suitable random number is 'guessed'.

The word 'guess' is used above as this would be an extremely poor design: it might take many attempts for an zero entry to be chosen depending on how full the board is, and it is theoretically possible (although extremely unlikely) for numbers representing occupied entries on the board to continuously be chosen, causing the algorithm never to halt. This is clearly not ideal

Instead, an alternative algorithm was used: as the number of zeroes is known (it is a state variable), a number between one and *this* number can be chosen instead. Denoting this number as i, the board is then stepped through until the ith zero is found, which is changed to a two. In its worst case performance, this algorithm may involve stepping through the entire board (if the number randomly chosen represents the last zero on the board, and this last zero is in the bottom right of the board), whereas the previous algorithm may require only one 'guess' to find a suitable entry, but this small downside is considerably overshadowed by the fact that *the first algorithm may never terminate*, whereas this algorithm is guaranteed to.

5.2 Row/Column Slide

The algorithm representing the shifting of a general row to the left is not particularly complex: the implementation chosen simply involves comparing subsequent pairs of non-zero entries on each row and moving them to one end of the column, if necessary merging them, along with some conditions to ensure that the algorithm does not try to continue past the end of the row. Note that simply moving the entries on a row around *does not change the number of zeroes.* This means that only when a merge operation is carried out and an extra zero is created must we update the variable holding the number of zeroes by incrementing it.

It is however that separate algorithms representing a shift left, right, up or down do not have to implemented: notice that once we have an algorithm that can to shift a general row to the left, it can be easily made to shift the row to the right by 'translating' how it indexes entries on the row: if the zero indexed variable, x, representing the position of an entry in the row is 'translated' in some way to $L - x$, where L is the index of the last entry in the row, then the algorithm will instead cause the row to be shifted to the right.[1]

[1] Sorry if this is slightly hand wavy.

A 'translation' or 'parametrisation' function is defined which 'translates' from the index of an entry on a general row on which the algorithm can run to the index of an entry in the vector representing the board, depending on whether the algorithm should operate on a column or a row, and in which direction. All motions are now possible, using the same algorithm which can on paper can only operate on a general row to shift all the elements to the right: if a shift up is required, the algorithm is run with the parametrisation function setup to translate to indexes representing a column within the matrix; a shift down does the same, but the parametrisation function reverses its output in a similar manner to that mentioned in the previous paragraph.

When vertical motion is required, the general algorithm is run on all the columns with the parametrisation function correctly set; horizontal motion causes the general algorithm to be run on all the rows wit the parametrisation function set another way.

5.3 Move Legality/End Game Detection

Move legality is checked by storing a copy of the game board before a move is made and comparing it to the board after the move: if there is a difference, the move was legal.

End game detection can be done by checking whether each move is legal after a move is played. If no moves are legal, then the game is over. If a move is possible, the game state should be immediately restored to how it was before the test was carried out: testing the legality of moves involves carrying out the move, but checking if the game is over should have no side effects. This can be optimised slightly by noticing that if there is a zero present on the board, then there is always a legal move (the converse is *not* true), so before checking each move for legality the number of zeroes is checked: if it is non-zero, it can be immediately determined that the game is over without having to check each move for legality.

6 Further Implementation Details and Rationale

6.1 Random Number Generation

c++11 features were used for random number generation, as they allow for uniform integer distributions to be easily created, avoiding the bias that often occurs when using a `rand() % x` method.

6.2 Enums

Enums were chosen to represent the status of the `Board` object, and the direction of a move, as they have a clear meaning, allowing the code to be easily inspected and understood.

6.3 `getStatus()`

In addition to the functions mentioned in Section 3, a public member function `getStatus()` was defined. This returns an enum representing the status of the `Board` object (whether it is normal, or whether it has reverted to a default configuration due to an invalid input file).

6.4 Further Functional Decomposition

In addition to functions previously mentioned, a function called `shiftAll()` was defined. This causes the general row shift algorithm to be called as many times as necessary to perform a move, removing some repetition of code from within the `move()` function.